Hibernate Query Language (HQL), Native Query and Criteria Query

This document explains Hibernate Query Language (HQL), Java Persistence Query Language (JPQL), Native Query, and Criteria Query in a simple, human-written style suitable for assignments and record submissions.

Hibernate Query Language (HQL)

Hibernate Query Language (HQL) is an object-oriented query language provided by Hibernate. It is similar to SQL but works with Java entity classes instead of database tables. HQL is database independent and allows developers to retrieve, update, and delete data using Java objects.

Example:
@Query("SELECT e FROM Employee e")

JPQL

Java Persistence Query Language (JPQL) is the standard query language defined by JPA. It is portable across different JPA providers. Every JPQL query is valid HQL, but HQL provides additional Hibernate-specific features.

Comparison of HQL and JPQL

• HQL is Hibernate-specific while JPQL is JPA standard.
• HQL supports INSERT queries whereas JPQL does not.
• JPQL is more portable.
• Both use entity classes instead of tables.

@Query Annotation

The @Query annotation in Spring Data JPA is used to write custom queries.

Example:
@Query("SELECT e FROM Employee e WHERE e.department.id=:id")

HQL Fetch Keyword

The FETCH keyword retrieves related objects together with the parent entity. It reduces unnecessary SQL queries and avoids LazyInitializationException.

Example:
SELECT e FROM Employee e LEFT JOIN FETCH e.department LEFT JOIN FETCH e.skillList WHERE e.permanent=1

Aggregate Functions

HQL supports AVG(), SUM(), COUNT(), MAX(), and MIN().
Example:
@Query("SELECT AVG(e.salary) FROM Employee e WHERE e.department.id=:id")

Native Query

A Native Query uses SQL directly instead of HQL.

Example:
@Query(value="SELECT * FROM employee", nativeQuery=true)
Advantages: Database-specific features, complex SQL.
Disadvantages: Less portable.

nativeQuery Attribute

Setting nativeQuery=true tells Spring Data JPA that the query is written in SQL instead of JPQL/HQL.

Criteria Query

Criteria Query is used to build queries dynamically using Java code. It is useful when search conditions change based on user input, such as e-commerce filters.

Main Components:
• CriteriaBuilder – creates query objects.
• CriteriaQuery – represents the query.
• Root – represents the entity.
• Predicate – represents conditions.
• TypedQuery – executes the query.

Example

CriteriaBuilder cb = entityManager.getCriteriaBuilder();
CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
Root<Employee> root = cq.from(Employee.class);
cq.select(root);
TypedQuery<Employee> query = entityManager.createQuery(cq);

Conclusion

HQL and JPQL simplify database interaction using Java objects. Native Query is useful for database-specific SQL, while Criteria Query is the preferred approach for creating dynamic queries. Together, these technologies make Spring Data JPA applications flexible, efficient, and easy to maintain.